

COMP101

Introduction to Programming

2025-26

Lecture-23

List Operations

List passing to functions

- ‘pass-by-reference’
 - original data in the list is passed to a function
 - function can change the original list
- ‘pass by copy’ (shallow copy and deep copy)
 - make a copy of the list ... then ...
 - pass the copy to the function
 - function can change the copy, not the original

shallow copy and side-effect

```
originalList = [1,2,3]  
copyList = originalList
```

```
print (originalList)
```

```
[1, 2, 3]
```

```
print (copyList)
```

```
[1, 2, 3]
```

originalList is copied and results
look okay ... but ...

```
originalList = [1,2,3]  
copyList = originalList  
copyList[2] = 4 #change copy
```

```
print (originalList)
```

```
[1, 2, 4] #changes original
```

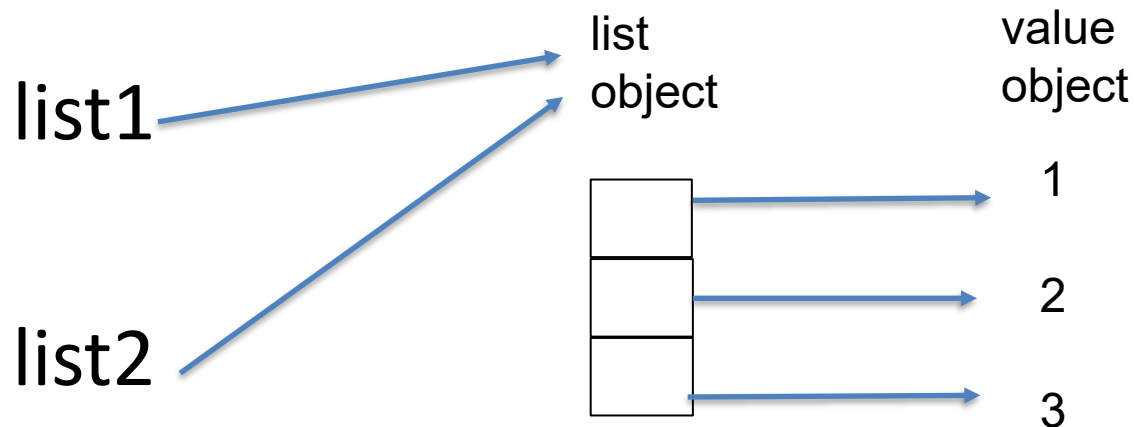
```
print (copyList)
```

```
[1, 2, 4] #changes the copy
```

... this is a side-effect of doing a
'shallow copy' – both lists change

shallow copy mechanism

- list1 points to an object in memory that holds three integers
- list2 is a 'shallow copy' of list1, so it points to the same object in memory



shallow copy mechanism

- A shallow copy makes the copied list point to the same list as the original list
- Change the original, we change the copy
- Change the copy, we change the original

`copy.copy()` to do a shallow copy

```
import copy
```

```
originalList = [list data]
```

```
deepList = copy.deepcopy(originalList)
```

Use to copy a top-level structure (no nested lists)
and okay to make changes to both lists

shallow copy no side-effect (slice)

#shallow copy side-effect

```
originalList = [1,2,3]
```

```
copyList = originalList
```

#shallow copy changes copyList to [1,2,4]

```
copyList[2] = 4
```

#shallow copy also changes the original

```
print (originalList)
```

```
[1, 2, 4]
```

```
print (copyList)
```

```
[1, 2, 4]
```

#use slice operator ':'

```
originalList = [1,2,3]
```

```
copyList = originalList[:] #slice
```

```
copyList [2] = 4 #changes copy
```

```
print (originalList)
```

```
[1, 2, 3] #no change to original
```

```
print (copyList)
```

```
[1, 2, 4]
```

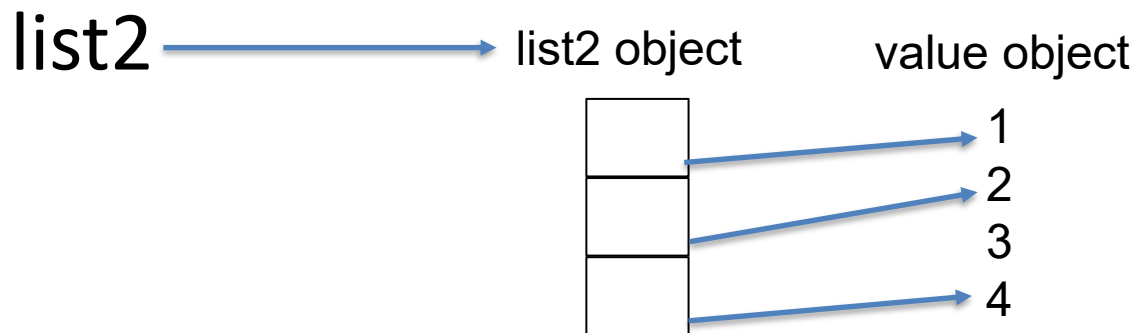
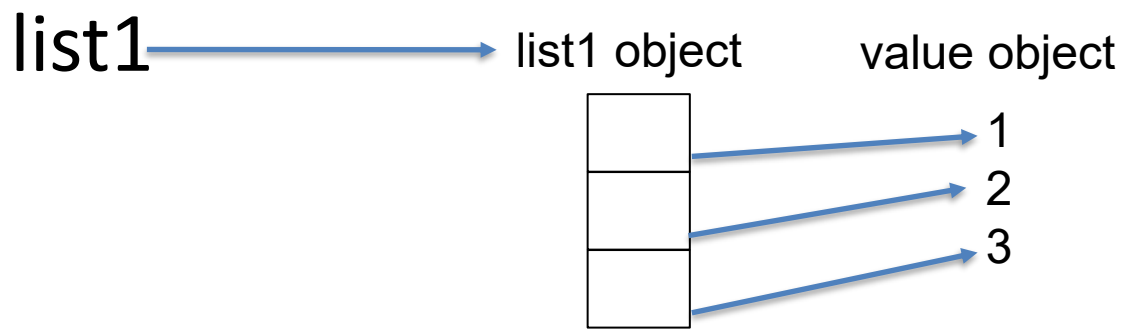
deep copy

- Copy with slice `[:]` makes the copied list point to its own list, not the original list
- Change the original, we don't change the copy
- Change the copy, we don't change the original
- 'deepcopy' is used for nested sub-lists, i.e. lists that contain lists and we don't want to change the original

Deep copyno side-effect with [:]

- Slice makes lists point to separate copies

`list2[2] = 4`



`copy.deepcopy ()` to do a deep copy

```
import copy
```

```
originalList = [list data]
```

```
shallowList = copy.copy(originalList)
```

List deletion - del[:] and del

- Delete all content with 'del' and 'slice' [:]

```
del spectrumList[:]
print (spectrumList)
[]
```

del list [:] **#uses a slice**
Slice operator deletes entire content;
Content can't be retrieved
**List still exists but is empty –
elements have been deleted**

- Delete the list by 'del'

```
del spectrumList
print (spectrumList)
NameError
```

del list **#note no slice**
Deletes content AND the list
**List no longer exists -
list AND elements have been deleted**

List reverse - .reverse()

```
list1 = [1,2,3]
print (list1)
[1,2,3]
```

```
list1.reverse()
print (list1)
[3,2,1]
```

#lists can be reversed – reverse a concatenated list

```
listNum = [1, 2, 3]
listStr = ["a","b","c"]
```

```
combined = listNum + listStr
print(combined)
[1, 2, 3, 'a', 'b', 'c']
```

```
combined.reverse()
print (combined)
['c', 'b', 'a', 3, 2, 1]
```

List reverse – stays reversed

```
sentence_list = ["This", "is", "a", "sentence"]  
sentence_list.reverse()  
print (sentence_list)  
['sentence', 'a', 'is', 'This']
```

when a list has been
reversed – it stays reversed

```
sentence_list.reverse()  
print (sentence_list)  
['This', 'is', 'a', 'sentence']
```

We can always reverse it
back to the way it was
...or...

List reverse – re-reverse it

- ... or ...make a copy and reverse the copy

```
originalList = ["This", "is", "a", "sentence"]
```

```
#copy the original list with [:] – now two separate lists
```

```
copyOfOriginal = originalList[:]
```

```
#reverse the copy – remember it is in the same order as the original at this moment
```

```
copyOfOriginal.reverse()
```

```
#o/p the copy, not the original
```

```
print (copyOfOriginal)
```

```
['sentence', 'a', 'is', 'This']
```

We now have two lists – use the right one

List sorting - .sort()

```
list1 = [2,1,3]
```

```
list1.sort()
```

```
print(list1)
```

```
[1, 2, 3]
```

The list has been sorted – and stays sorted.
Make a copy and work with the copy if you
want the original

```
list2 = ["dog", "cat", "bat"]
```

```
list2.sort()
```

```
print (list2)
```

```
['bat', 'cat', 'dog']
```

List sorting – uppercase values

```
sentence_list = ["This","is","a", "sentence"]
```

```
sentence_list.sort()
```

```
print(sentence_list)
```

```
['This', 'a', 'is', 'sentence']
```

Capitals sorted first with priority over lower case strings e.g.

```
sentence_list = ["This","is","a","Big","sentence"]
```

```
sentence_list.sort()
```

```
['Big', 'This', 'a', 'is', 'sentence']
```